

Effektivitet i Scala: Sorteringsalgoritmer

Fiktiv Författare

12 september 2024

Notera

Denna artikel är primärt genererad med hjälp av AI-verktyg, och sedan manuellt kontrollerad och justerad. Alla data och mätningar är påhittade för övningsändamål.

1 Introduktion

Scala är ett populärt språk som kombinerar funktionell och objektorienterad programmering. I denna artikel utvärderar vi effektiviteten hos olika sorteringsalgoritmer när de används i Scala. Speciellt fokuserar vi på sorteringsalgoritmerna *QuickSort* och *MergeSort*.

2 O-notation

Innan vi börjar prata om algoritmers tidseffektivitet behöver vi förstå så kallad *O-notation*. O-notation (ibland kallad "stora O-notation" eller "big O notation" på engelska) är ett sätt att beskriva hur effektiv en algoritm är när det gäller exekveringstid, beroende på mängden data som algoritmen hanterar – så kallad *tidskomplexitet*. Den används ofta inom datavetenskap för att klassificera algoritmer baserat på deras prestanda, särskilt för stora inmatningsstorlekar.

O-notationen uttrycker hur algoritmens tid växer som en funktion av storleken på indata, vanligtvis betecknad som n . Till exempel, om en algoritm har en tidskomplexitet på $O(n)$, betyder det att tiden det tar att utföra algoritmen ökar linjärt med storleken på indata. O:et uttalas ofta som "ordo", alltså "ordo n " för $O(n)$ eller "ordo $n \log n$ " för $O(n \log n)$.

2.1 Vanliga O-klasser

Här är några av de vanligaste O-notationerna och vad de representerar:

- $O(1)$ Konstant tid – Algoritmen tar samma tid oavsett storleken på indata.
- $O(n)$ Linjär tid – Tiden ökar direkt proportionellt mot storleken på indata. En dubbling av indata leder till en dubbling av exekveringstiden.
- $O(\log n)$ Logaritmisk tid – Tiden ökar mycket långsamt när indata växer. Vanligt i effektiva sökalgoritmer.
- $O(n \log n)$ Linjär-logaritmisk tid – Vanligt för många effektiva sorteringsalgoritmer, som exempelvis MergeSort.
- $O(n^2)$ Kvadratisk tid – Tiden växer kvadratiskt med indata; ofta sett i enklare sorteringsalgoritmer som Bubble Sort. En dubbling av indata leder till en fyrdubbling av exekveringstiden.
- $O(2^n)$ Exponentiell tid – Tiden växer mycket snabbt med storleken på indata; vanligt i vissa rekursiva algoritmer.

2.2 Exempel

Anta att vi har en sorteringsalgoritm som tar en lista med n tal och jämför varje tal med varje annat tal för att avgöra ordningen. Denna algoritm skulle ha en tidskomplexitet på $O(n^2)$ eftersom varje element (n stycken) jämförs med alla andra element (n stycken)¹.

O-notationen hjälper oss att förstå och förutsäga hur algoritmer beter sig när datamängden växer, och är därför ett ovärderligt verktyg vid val av algoritmer och datastrukturer.

3 Sorteringsalgoritmer

Sortering är en grundläggande operation inom datavetenskap, och olika algoritmer har olika prestanda beroende på data och sammanhang. Två av de vanligaste algoritmerna är *QuickSort* och *MergeSort*.

¹Produkten blir egentligen $n \cdot (n - 1)$, men konstanta värden ignoreras i O-notation eftersom de inte spelar någon roll när n går mot ett stort värde.

3.1 Jämförelse av QuickSort och MergeSort

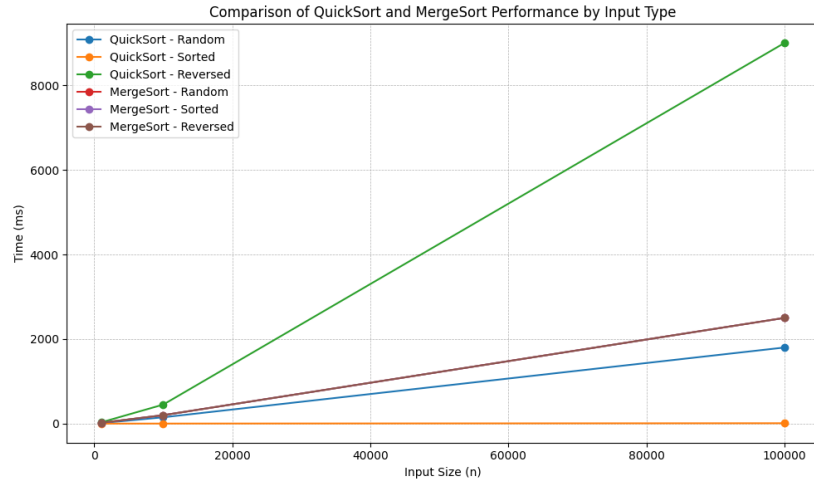
QuickSort är en snabb sorteringsalgoritm som fungerar bra i genomsnittsfallet med tidskomplexiteten $O(n \log n)$. Den kan dock prestera dåligt i det värsta fallet, med en tidskomplexitet på $O(n^2)$, om pivotvalet är ogynnsamt. MergeSort, å andra sidan, har alltid en tidskomplexitet på $O(n \log n)$, vilket gör den stabilare men ibland långsammare på grund av dess behov av extra minnesutrymme.

I tabell 1 nedan jämförs prestandan hos QuickSort och MergeSort för olika indata. Se Appendix A för implementationen i Scala. (*Notera igen att datan är påhittad för övningsändamål.*)

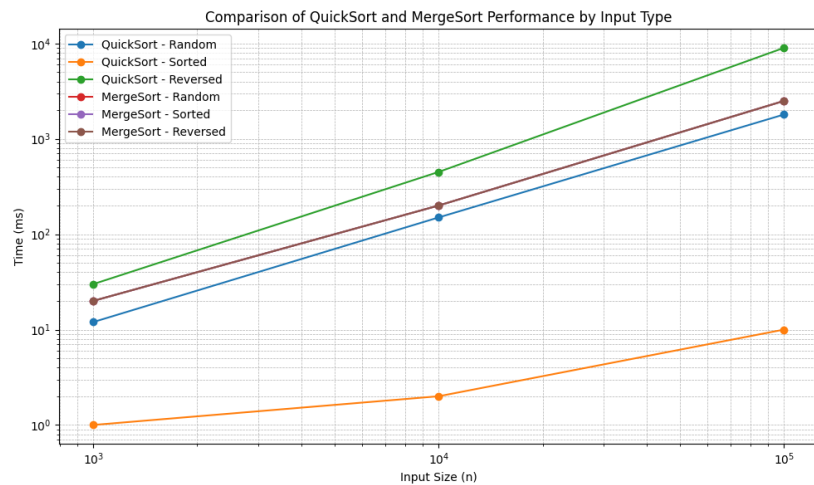
Algoritm	Indatastorlek (n)	Indatatyp	Tid (μs)	Minne (MB)
QuickSort	1000	Slumpmässig	12	5
QuickSort	1000	Sorterad	1	5
QuickSort	1000	Omvänd	30	5
QuickSort	10000	Slumpmässig	150	5
QuickSort	10000	Sorterad	2	5
QuickSort	10000	Omvänd	450	5
QuickSort	100000	Slumpmässig	1800	5
QuickSort	100000	Sorterad	10	5
QuickSort	100000	Omvänd	9000	5
MergeSort	1000	Slumpmässig	20	10
MergeSort	1000	Sorterad	20	10
MergeSort	1000	Omvänd	20	10
MergeSort	10000	Slumpmässig	200	10
MergeSort	10000	Sorterad	200	10
MergeSort	10000	Omvänd	200	10
MergeSort	100000	Slumpmässig	2500	10
MergeSort	100000	Sorterad	2500	10
MergeSort	100000	Omvänd	2500	10

Tabell 1: Jämförelse av QuickSort och MergeSort för olika indatatyper och -storlekar i Scala. Tid mäts i microsekunder och minnesanvändning i Megabyte. "Omvänd" betyder att datan är sorterad i omvänd ordning, vilket är värstafallet för QuickSort.

Samma data presenteras grafiskt i figur 1 och 2.



Figur 1: Exekveringstid för de två sorteringsalgoritmerna QuickSort och MergeSort i Scala. För båda algoritmerna plottas tre olika indatatyper separat: Slumpmässig, sorterat och omvänt sorterad indata. **(Linjär skala)**



Figur 2: Exekveringstid för de två sorteringsalgoritmerna QuickSort och MergeSort i Scala. För båda algoritmerna plottas tre olika indatatyper separat: Slumpmässig, sorterat och omvänt sorterad indata. **(Logaritmisk skala)**

4 Matematisk analys av algoritmer

Den matematiska analysen av dessa algoritmer kan göras med hjälp av deras tidskomplexitet. För QuickSort är den förväntade tidskomplexiteten:

$$T_Q(n) = O(n \log n)$$

Medan för MergeSort har vi:

$$T_M(n) = O(n \log n)$$

De har alltså samma tidskomplexitet i det genomsnittliga fallet. Trots liknande teoretisk prestanda kan dock det praktiska utförandet variera beroende på storlek och typ av indata, som vi såg i tabell 1 och figurerna 1 och 2.

5 Slutsats

Vi har undersökt två olika sorteringsalgoritmer i Scala och jämfört deras prestanda. Resultaten visar att valet av algoritm kan ha en betydande påverkan på programmets effektivitet. Genom att förstå dessa skillnader kan utvecklare fatta mer informerade beslut när de designar och optimerar sina program.

A Implementation i Scala

I kodlistning 1 nedan är ett exempel på en implementation av QuickSort och MergeSort i Scala:

Listing 1: Scala-program med implementation av QuickSort och MergeSort.

```
1 object SortComparison:
2   def quickSort(arr: Array[Int]): Array[Int] =
3     if arr.length <= 1 then arr
4     else
5       val pivot = arr(arr.length / 2)
6       Array.concat(
7         quickSort(arr.filter(_ < pivot)),
8         arr.filter(_ == pivot),
9         quickSort(arr.filter(_ > pivot))
10      )
11
12   def mergeSort(arr: Array[Int]): Array[Int] =
13     if arr.length <= 1 then arr
14     else
15       val (left, right) = arr.splitAt(arr.length / 2)
16       val sortedLeft = mergeSort(left)
17       val sortedRight = mergeSort(right)
18       merge(sortedLeft, sortedRight)
19
20   private def merge(left: Array[Int], right: Array[Int]): Array[Int] =
21     (left, right) match
22       case (Array(), _) => right
23       case (_, Array()) => left
24       case (l, r) =>
25         if l.head < r.head then l.head +: merge(l.tail, r)
26         else r.head +: merge(l, r.tail)
```